

Greedy Set Cover

Zweiphasiger Algorithmus zur precision-optimalen Regelkomposition

1 Problemstellung

Gegeben sei ein Datensatz mit n Zeilen, m binären Prädikaten $p_1, \dots, p_m$ (aufgeteilt in Feature-Gruppen) sowie einem binären Label $\mathbf{y} \in \{0, 1\}^n$ mit $\text{total_pos} = \mathbf{1}^\top \mathbf{y}$ Positiven.

Ziel: Finde eine *Composition*

$$C = R_1 \vee R_2 \vee \dots \vee R_r$$

aus r AND-Regeln, sodass die **Precision** maximiert wird, während ein gewünschter **Recall** erreicht wird. Im Unterschied zu den B&B-Ansätzen wird kein fester min_recall -Constraint gesetzt. Stattdessen liefert der Algorithmus *alle Zwischenschritte* — eine monoton steigende Recall-Folge — aus der für jedes gewünschte Recall-Ziel die beste Composition abgelesen werden kann.

Parameter:

- **max_k:** Maximale Anzahl Prädikate pro AND-Regel
- **min_tp:** Minimale True Positives für Kandidatenregeln (Phase 1)
- **min_new_tp:** Minimale *neue* True Positives, damit eine Regel in einem Schritt ausgewählt wird (Phase 2)
- **max_candidates:** Obergrenze für die Kandidatenanzahl (Diversity-Filter)

2 Phase 1: Kandidatengenerierung (Apriori)

Die Kandidatengenerierung nutzt denselben Apriori-Algorithmus wie der Apriori+B&B-Ansatz. Die Anti-Monotonie der AND-Verknüpfung dient als Pruning-Kriterium:

$$TP(R \wedge p) \leq TP(R)$$

2.1 Level-weiser Aufbau

Level 1: Erzeuge alle Ein-Prädikat-Regeln $\{p_i\}$ mit $TP(p_i) \geq \text{min_tp}$.

Level $\ell \rightarrow \ell + 1$ (bis $\ell = \text{max_k}$): Für jede Regel R aus Level ℓ und jedes Prädikat p_j mit Gruppenindex $>$ höchster Gruppenindex in R :

- Bilde $R' = R \wedge p_j$
- Falls $TP(R') \geq \text{min_tp}$: behalte R'
- Sonst: verwerfe (Pruning)

Die Constraint *maximal ein Prädikat pro Feature-Gruppe* wird durch den aufsteigenden Gruppenindex sichergestellt.

2.2 Scoring und Diversity-Filter

Jede Kandidatenregel R_i wird durch ihren Maskenvektor $\mathbf{m}_i \in \{0,1\}^n$ repräsentiert und bekommt zwei Scores:

$$prec_i = \frac{\mathbf{1}^\top (\mathbf{m}_i \wedge \mathbf{y})}{\mathbf{1}^\top \mathbf{m}_i}, \quad tp_i = \mathbf{1}^\top (\mathbf{m}_i \wedge \mathbf{y})$$

Falls die Kandidatenanzahl $|\mathcal{R}|$ die Obergrenze `max_candidates` überschreitet, greift ein **Diversity-Filter**: Die Top- $N/2$ nach Precision und die Top- $N/2$ nach TP werden vereinigt. Dadurch bleiben sowohl präzise als auch recall-starke Regeln erhalten.

3 Phase 2: Greedy-Selektion

3.1 Grundidee

Der Greedy-Algorithmus baut die Composition schrittweise auf: In jedem Schritt wird genau eine Regel ausgewählt und per OR zur bestehenden Composition hinzugefügt. Das Auswahlkriterium ist die **marginale Precision** — die Precision der *gesamten* Composition nach Hinzufügen der neuen Regel.

3.2 Formale Beschreibung

Sei $\mathbf{c}^{(t)}$ der OR-Vektor der Composition nach Schritt t :

$$\mathbf{c}^{(0)} = \mathbf{0}, \quad \mathbf{c}^{(t)} = \mathbf{c}^{(t-1)} \vee \mathbf{m}_{s_t}$$

wobei s_t die in Schritt t gewählte Regel ist. Definiere:

$$\begin{aligned} tp^{(t)} &= \mathbf{1}^\top (\mathbf{c}^{(t)} \wedge \mathbf{y}) && \text{(kumulative True Positives)} \\ pos^{(t)} &= \mathbf{1}^\top \mathbf{c}^{(t)} && \text{(kumulative Positives)} \end{aligned}$$

Für eine Kandidatenregel R_i mit Maske $\mathbf{m}_i$ berechne die *neuen, noch nicht abgedeckten* Treffer:

$$\begin{aligned} \mathbf{u}_i &= \mathbf{m}_i \wedge \neg \mathbf{c}^{(t-1)} && \text{(uncovered = neue Treffer durch } R_i) \\ \Delta tp_i &= \mathbf{1}^\top (\mathbf{u}_i \wedge \mathbf{y}) && \text{(neue True Positives)} \\ \Delta pos_i &= \mathbf{1}^\top \mathbf{u}_i && \text{(neue Positives insgesamt)} \end{aligned}$$

Die **Auswahlregel** in Schritt t ist:

$$s_t = \arg \max_{i \in \mathcal{A}^{(t)}} \frac{tp^{(t-1)} + \Delta tp_i}{pos^{(t-1)} + \Delta pos_i}$$

unter der Nebenbedingung $\Delta tp_i \geq \text{min_new_tp}$.

3.3 Alive-Menge und Terminierung

Die Menge der “lebenden” Kandidaten $\mathcal{A}^{(t)}$ wird in jedem Schritt aktualisiert:

- Falls $\Delta tp_i < \text{min_new_tp}$: Regel R_i wird **permanent entfernt**. Da die OR-Maske $\mathbf{c}$ in jedem Schritt nur wächst, ist Δtp_i monoton fallend — eine Regel, die jetzt unter dem Schwellwert liegt, kann ihn später nie mehr erreichen.

Der Algorithmus terminiert, wenn:

1. Keine Regel in $\mathcal{A}^{(t)}$ hat $\Delta tp_i \geq \text{min_new_tp}$, oder
2. `max_steps` erreicht ist.

4 Algorithmus

Algorithm 1 Greedy Set Cover

Require: Kandidatenregeln $\{(R_i, \mathbf{m}_i)\}$, Labels $\mathbf{y}$, Parameter `min_new_tp`

Ensure: Schrittfolge $\mathcal{S} = [(t, recall^{(t)}, prec^{(t)}, Regeln^{(t)})]$

```

1:  $\mathbf{c} \leftarrow \mathbf{0}^n$ ,  $tp \leftarrow 0$ ,  $pos \leftarrow 0$ 
2:  $\mathcal{A} \leftarrow \{1, \dots, |\mathcal{R}|\}$  ▷ Alive-Menge
3: for  $t = 1, 2, \dots$  do
4:    $best\_score \leftarrow -1$ ,  $best \leftarrow \text{nil}$ 
5:    $\mathcal{A}' \leftarrow \emptyset$ 
6:   for  $i \in \mathcal{A}$  do
7:      $\mathbf{u}_i \leftarrow \mathbf{m}_i \wedge \neg \mathbf{c}$ 
8:      $\Delta tp_i \leftarrow \mathbf{1}^\top (\mathbf{u}_i \wedge \mathbf{y})$ 
9:     if  $\Delta tp_i < \text{min\_new\_tp}$  then
10:      continue ▷ Permanent entfernt ( $\Delta tp$  monoton fallend)
11:    end if
12:     $\mathcal{A}' \leftarrow \mathcal{A}' \cup \{i\}$ 
13:     $\Delta pos_i \leftarrow \mathbf{1}^\top \mathbf{u}_i$ 
14:     $score_i \leftarrow (tp + \Delta tp_i) / (pos + \Delta pos_i)$ 
15:    if  $score_i > best\_score$  then
16:       $best\_score \leftarrow score_i$ ,  $best \leftarrow i$ 
17:    end if
18:  end for
19:   $\mathcal{A} \leftarrow \mathcal{A}'$ 
20:  if  $best = \text{nil}$  then
21:    break ▷ Keine Regel erfüllt min_new_tp
22:  end if
23:   $\mathbf{c} \leftarrow \mathbf{c} \vee \mathbf{m}_{best}$ 
24:   $tp \leftarrow tp + \Delta tp_{best}$ ,  $pos \leftarrow pos + \Delta pos_{best}$ 
25:  Speichere Schritt  $(t, tp/\text{total\_pos}, tp/pos, Regeln)$ 
26: end for

```

5 Beispiel

Gegeben seien 4 Kandidatenregeln mit folgenden Maskenvektoren ($n = 10$, $\text{total_pos} = 5$, $\text{min_new_tp} = 1$):

	R_1	R_2	R_3	R_4
Maske $\mathbf{m}_i$	1100100000	0011010000	0000011100	0000000011
TP	2	2	1	0
Pos	3	3	3	2
Precision	66.7%	66.7%	33.3%	0%

Label: $\mathbf{y} = 1101010000$ (Positives an Stellen 1, 2, 4, 6).

Schritt 1: Alle Regeln werden evaluiert. R_4 hat $TP = 0$ und wird entfernt. R_1 und R_2 haben marginale Precision $\frac{2}{3} = 66,7\%$. R_1 wird gewählt (Tie-Break).

$$\mathbf{c}^{(1)} = 1100100000, \quad tp = 2, \quad pos = 3, \quad \text{Recall} = \frac{2}{5} = 40\%$$

Schritt 2: Nur noch uncovered Treffer zählen. R_2 : $\mathbf{u}_2 = 0011010000$, $\Delta tp = 2$, $\Delta pos = 3$. Marginale Precision: $\frac{2+2}{3+3} = 66,7\%$. R_3 : $\Delta tp = 1$, $\Delta pos = 3$, marginale Precision: $\frac{2+1}{3+3} = 50\%$. $\Rightarrow R_2$ wird gewählt.

$$\mathbf{c}^{(2)} = 1111110000, \quad tp = 4, \quad pos = 6, \quad \text{Recall} = 80\%$$

Schritt 3: R_3 : $\mathbf{u}_3 = 0000001100$, $\Delta tp = 0 \Rightarrow$ entfernt. Keine Kandidaten mehr. **Terminierung.**

Ergebnis: 2 Regeln, Recall = 80%, Precision = 66,7%. Für Recall $\geq 40\%$ wählt man Schritt 1 (1 Regel, 66,7%), für $\geq 80\%$ Schritt 2 (2 Regeln, 66,7%).

6 Eigenschaften

6.1 Determinismus und Monotonie

Der Algorithmus ist **vollständig deterministisch**: Bei gleichen Eingaben und Parametern ergibt sich stets dieselbe Schrittfolge. Da in jedem Schritt mindestens $\text{min_new_tp} \geq 1$ neue True Positives hinzukommen, ist die Recall-Folge **streng monoton steigend**:

$$\text{recall}^{(0)} = 0 < \text{recall}^{(1)} < \dots < \text{recall}^{(T)}$$

Die Precision-Folge ist dagegen im Allgemeinen **monoton fallend**, da spätere Regeln typischerweise unpräziser sind.

6.2 Einmaliges Laufen statt mehrfacher Optimierung

Da der Algorithmus deterministisch ist, genügt **ein einziger Lauf** bis zur Terminierung. Aus der gespeicherten Schrittfolge kann für jedes beliebige Recall-Ziel r^* die optimale Composition abgelesen werden:

$$\text{Schritt}^*(r^*) = \min\{t : \text{recall}^{(t)} \geq r^*\}$$

Dies ist wesentlich effizienter als separate Läufe für verschiedene Recall-Ziele.

6.3 Rolle von min_new_tp

Der Parameter min_new_tp steuert den Trade-off zwischen Präzision und Recall-Abdeckung:

min_new_tp	Auswirkung
= 1	Jede Regel mit mindestens einem neuen TP wird berücksichtigt. Maximaler erreichbarer Recall, aber viele unpräzise Regeln in späteren Schritten.
= 5	Balance: genügend Granularität bei akzeptabler Regelqualität.
= 10	Nur Regeln mit substanziellem Beitrag werden gewählt. Weniger Schritte, aber Recall-Lücken möglich.

6.4 Adaptive Schwelle

In der Variante mit festem Recall-Ziel (**greedy_select**) wird min_new_tp nicht konstant gehalten, sondern **adaptiv** an den verbleibenden Bedarf angepasst:

$$\text{adaptive_min}^{(t)} = \max\left(\text{min_new_tp}, \lfloor 0,01 \cdot (\text{min_tp} - tp^{(t-1)}) \rfloor\right)$$

wobei $\text{min_tp} = \lceil \text{min_recall} \cdot \text{total_pos} \rceil$ die Mindestanzahl an TPs ist.

Intuition: Am Anfang fehlen noch viele TPs (z. B. $\text{remaining} = 2.800$), sodass 1% davon = 28 ist — nur Regeln mit substanziellem Beitrag werden gewählt. Gegen Ende ($\text{remaining} = 50$) sinkt die Schwelle auf $\max(\text{min_new_tp}, 0) = \text{min_new_tp}$, sodass auch kleine Regeln zugelassen werden, um den letzten Recall zu erreichen.

Konsequenz für die Alive-Menge: Da die adaptive Schwelle sinken kann, dürfen Regeln mit $\Delta tp_i < \text{adaptive_min}^{(t)}$ nicht permanent entfernt werden — sie könnten in einem späteren Schritt mit niedrigerer Schwelle wieder in Frage kommen. Hier wird nur bei $\Delta tp_i < 1$ permanent entfernt (anders als bei der Variante mit fixem min_new_tp).

Effekt: Frühe Schritte wählen breite, recall-starke Regeln; späte Schritte füllen mit kleineren, präzisen Regeln auf. Dadurch entsteht ein natürlicher Übergang von grober zu feiner Abdeckung.

7 Vergleich mit B&B-Ansätzen

	Greedy Set Cover	B&B-Ansätze
Optimalität	Greedy-Approximation (keine Garantie für globales Optimum)	Exakt optimal (innerhalb des Suchraums)
Laufzeit	$O(T \cdot \mathcal{R} \cdot n)$ mit T Schritten	Exponentiell im Worst Case
Skalierbarkeit	Problemlos für Millionen Zeilen und Tausende Kandidaten	Begrenzt durch $\binom{R}{r}$ Kombinationen
Recall-Constraint	Implizit durch Schrittfolge	Explizit als Pruning-Kriterium
Mehrere Recall-Ziele	Ein Lauf für alle	Je ein separater Lauf

8 Komplexität

Phase	Komplexität	Beschreibung
Apriori	$O(m^k \cdot n)$	Level-weiser Aufbau, m Prädikate, k Levels
Scoring	$O(\mathcal{R} \cdot n)$	Einmalige Precision/TP-Berechnung
Greedy	$O(T \cdot \mathcal{R} \cdot n)$	T Schritte, je $ \mathcal{R} $ Kandidaten

Praktisch: Die Alive-Menge $\mathcal{A}$ schrumpft in jedem Schritt, sodass spätere Schritte schneller sind. Für $k = 3$, $|\mathcal{R}| \approx 3.000$ Kandidaten und $n = 2,8\text{M}$ Zeilen dauert ein vollständiger Lauf mit ~ 70 Schritten wenige Sekunden.

9 Nachverarbeitung

Nach der Greedy-Selektion enthält die Composition möglicherweise redundante Regeln. Drei Nachverarbeitungsschritte reduzieren die Regelanzahl, ohne den Recall zu verringern (bei Subsumption und Unique-TP) bzw. mit kontrolliertem Recall-Verlust (bei Merge).

9.1 Subsumption Removal

9.1.1 Threshold-Hierarchie

Prädikate derselben Feature-Gruppe bilden eine **Lockerheitshierarchie**. Strenge Schwellwerte matchen weniger Zeilen als lockere:

$$\text{eq} \subset \text{dist_1} \subset \text{dist_2} \subset \text{dist_3} \subset \text{dist_4}$$

$$\text{prop_0.99} \subset \text{prop_0.95} \subset \text{prop_0.9} \subset \text{prop_0.8} \subset \text{prop_0.7}$$

$$\text{overlap_0.75} \subset \text{overlap_0.5} \subset \text{overlap_0}$$

Jeder Schwellwert τ hat einen **Lockerheitsindex** $\ell(\tau) \in \mathbb{N}_0$, wobei $\ell = 0$ der strengste ist. Für zwei Schwellwerte τ_1, τ_2 derselben Familie mit $\ell(\tau_1) \leq \ell(\tau_2)$ gilt:

$$\mathbf{m}(\tau_1_f) \subseteq \mathbf{m}(\tau_2_f)$$

d. h. jede Zeile, die den strengen Schwellwert erfüllt, erfüllt auch den lockeren.

9.1.2 Dominanz-Relation

Eine Regel A **dominiert** eine Regel B (geschrieben $A \succ B$), wenn:

1. Für jedes Prädikat p_A in A existiert ein Prädikat p_B in B aus *derselben* Feature-Gruppe mit $\ell(p_A) \geq \ell(p_B)$ (d. h. A ist mindestens so locker wie B in dieser Gruppe).
2. B darf zusätzliche Prädikate besitzen (die B noch enger machen).

Falls $A \succ B$, dann gilt für die Masken: $\mathbf{m}_B \subseteq \mathbf{m}_A$. Jede Zeile, die von B abgedeckt wird, wird auch von A abgedeckt. In einer OR-Composition ist B daher **redundant**.

9.1.3 Beispiel

Gegeben seien drei ausgewählte Regeln:

$$R_1 = \{\text{dist_2_end_date}\}$$

$$R_2 = \{\text{eq_end_date}, \text{eq_start_date}\}$$

$$R_3 = \{\text{dist_1_end_date}, \text{overlap_0.5_X}\}$$

Prüfe $R_1 \succ R_2$: R_1 hat **dist_2_end_date** (Gruppe **end_date**, $\ell = 2$). R_2 hat **eq_end_date** in derselben Gruppe mit $\ell = 0 \leq 2$ ✓. R_2 hat ein zusätzliches Prädikat **eq_start_date** ✓. $\Rightarrow R_1 \succ R_2$: R_2 **wird entfernt**.

Prüfe $R_1 \succ R_3$: R_1 hat **dist_2_end_date** ($\ell = 2$). R_3 hat **dist_1_end_date** ($\ell = 1 \leq 2$) ✓. R_3 hat **overlap_0.5_X** zusätzlich ✓. $\Rightarrow R_1 \succ R_3$: R_3 **wird entfernt**.

Ergebnis: Von 3 Regeln bleibt nur R_1 übrig. Der Recall bleibt unverändert, da $\mathbf{m}_{R_2} \cup \mathbf{m}_{R_3} \subseteq \mathbf{m}_{R_1}$. Die Precision kann nur steigen (weniger FP durch weniger Regeln).

9.1.4 Algorithmus

Für jedes Regelpaar (R_i, R_j) mit $|R_i| \leq |R_j|$ wird geprüft, ob $R_i \succ R_j$. Falls ja, wird R_j als dominiert markiert. Komplexität: $O(r^2 \cdot k)$ mit r Regeln und maximal k Prädikaten pro Regel.

9.2 Unique-TP Removal

Nach der Subsumption-Bereinigung kann es Regeln geben, die zwar nicht von einer einzelnen anderen Regel dominiert werden, deren True Positives aber *insgesamt* bereits von der Vereinigung aller anderen Regeln abgedeckt sind.

9.2.1 Definition

Für eine Regel R_i in der Composition $\{R_1, \dots, R_r\}$ sind die **Unique True Positives**:

$$UTP(R_i) = \mathbf{1}^\top \left(\mathbf{m}_i \wedge \neg \bigvee_{j \neq i} \mathbf{m}_j \wedge \mathbf{y} \right)$$

Das sind die TPs, die *ausschließlich* von R_i abgedeckt werden und von keiner anderen Regel in der Composition.

Falls $UTP(R_i) = 0$: Entfernung von R_i ändert den Recall nicht (alle TPs von R_i werden von anderen Regeln abgedeckt), aber die Precision steigt, da die FPs von R_i wegfallen.

9.2.2 Beispiel

	Maske	TP-Maske	UTP
R_1	111000	110000	1 (Pos. 1)
R_2	011100	010100	1 (Pos. 4)
R_3	010010	010000	0

$\mathbf{y} = 110100$. R_3 hat TP an Position 2, aber R_1 und R_2 decken Position 2 ebenfalls ab. $\Rightarrow UTP(R_3) = 0$, R_3 wird entfernt. Recall bleibt bei $\frac{3}{3} = 100\%$, aber die FP von R_3 (Position 5) fällt weg.

9.2.3 Algorithmus

Für jede Regel R_i : Berechne die OR-Maske aller anderen Regeln $\bigvee_{j \neq i} \mathbf{m}_j$, dann $UTP(R_i)$. Falls $UTP(R_i) = 0$, entferne R_i . Komplexität: $O(r^2 \cdot n)$.

9.3 Merge

Der Merge-Schritt reduziert die Anzahl der Regeln durch **Zusammenfassung ähnlicher Regeln**. Regeln, die viele gemeinsame Prädikate teilen, werden zu einer einzigen, strengeren Regel vereinigt.

9.3.1 Vorgehen

1. **Paar-Häufigkeiten zählen**: Für jedes Prädikatpaar (p_a, p_b) wird gezählt, in wie vielen Regeln beide gleichzeitig vorkommen.
2. **Greedy-Gruppierung**: Das häufigste Paar wird ausgewählt. Alle Regeln, die dieses Paar enthalten, bilden eine Gruppe.
3. **Vereinigung**: Alle Prädikate der Gruppe werden zu einer einzigen AND-Regel zusammengefasst (Mengenvereinigung).
4. Wiederhole für das nächsthäufigste Paar (nur mit noch nicht gruppierten Regeln).

9.3.2 Beispiel

Gegeben seien 4 Regeln:

$$\begin{aligned} R_1 &= \{A, B, C\} \\ R_2 &= \{A, B, D\} \\ R_3 &= \{A, B, E\} \\ R_4 &= \{C, D, F\} \end{aligned}$$

Das Paar (A, B) kommt in R_1, R_2, R_3 vor (3-mal). Diese drei Regeln werden zusammengefasst:

$$R_1 \cup R_2 \cup R_3 = \{A, B, C, D, E\}$$

R_4 bleibt unverändert. **Ergebnis:** 2 Regeln statt 4.

9.3.3 Auswirkung auf Metriken

Der Merge ist **nicht recall-erhaltend**. Die ursprüngliche Composition

$$(A \wedge B \wedge C) \vee (A \wedge B \wedge D) \vee (A \wedge B \wedge E)$$

wird zu

$$A \wedge B \wedge C \wedge D \wedge E$$

Die zusammengefasste Regel verlangt *alle* Prädikate gleichzeitig (AND), während die ursprünglichen Regeln per OR verknüpft waren. Dadurch matcht die Merge-Regel *weniger* Zeilen:

$$\mathbf{m}_{A \wedge B \wedge C \wedge D \wedge E} \subseteq \mathbf{m}_{A \wedge B \wedge C} \cup \mathbf{m}_{A \wedge B \wedge D} \cup \mathbf{m}_{A \wedge B \wedge E}$$

Der Vorteil liegt in der **Interpretierbarkeit**: Weniger, aber längere Regeln sind oft leichter zu verstehen als viele kurze. Außerdem steigt typischerweise die Precision, da die Merge-Regel strenger filtert.

Anwendung: Der Merge wird nach der Greedy-Selektion und den recall-erhaltenden Bereinigungen (Subsumption, Unique-TP) durchgeführt. Recall und Precision werden danach neu berechnet.